

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN

MÔN: CÔNG NGHỆ PHẦN MỀM

XÂY DỰNG HỆ THỐNG CHATBOT HỖ TRỢ HỌC TẬP ỨNG DỤNG RAG

Giảng viên hướng dẫn: Thầy Nguyễn Văn Chiến

Nhóm thực hiện: Nhóm 9

1. Phan Liên Hưng - MSSV: 034206016528
2. Nguyễn Ninh Công Danh - MSSV: 079205006602
3. Nguyễn Hiễn Đạt - MSSV: 079206026928
4. Đào Văn Khánh - MSSV: 051206009944
5. Phạm Hoàng Dũng - MSSV: 070205007005
6. Lương Thành Đạt - MSSV: 056206003075
7. Trần Thị Kiều Trinh - MSSV: 066306009531

Thành phố Hồ Chí Minh, 2026

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án “Xây dựng hệ thống Chatbot hỗ trợ hỏi đáp tài liệu bằng công nghệ RAG”, nhóm chúng em đã nhận được rất nhiều sự hướng dẫn, giúp đỡ và tạo điều kiện thuận lợi từ phía Nhà trường, Khoa Công nghệ Thông tin cùng quý Thầy, Cô.

Đầu tiên, nhóm xin gửi lời cảm ơn chân thành đến Ban Giám hiệu Nhà trường cùng toàn thể quý Thầy, Cô thuộc Khoa Công nghệ Thông tin đã truyền đạt những kiến thức nền tảng cũng như những kinh nghiệm quý báu trong suốt quá trình học tập. Đây chính là tiền đề quan trọng giúp nhóm có đủ kiến thức để hoàn thành đồ án.

Đặc biệt, nhóm xin gửi lời cảm ơn sâu sắc đến Giảng viên hướng dẫn đã tận tình hướng dẫn, góp ý và hỗ trợ nhóm trong suốt quá trình nghiên cứu, phân tích yêu cầu, thiết kế hệ thống, triển khai chương trình cũng như hoàn thiện báo cáo. Những góp ý của Thầy/Cô đã giúp nhóm hoàn thiện sản phẩm cả về mặt kỹ thuật lẫn nội dung báo cáo.

Trong quá trình thực hiện, nhóm đã tìm hiểu và ứng dụng nhiều công nghệ hiện đại như Retrieval-Augmented Generation (RAG), ChromaDB, FastAPI, ReactJS, mô hình ngôn ngữ lớn (LLM), kỹ thuật Embedding văn bản và xây dựng API phục vụ hệ thống Chatbot. Thông qua quá trình này, nhóm không chỉ củng cố kiến thức chuyên môn mà còn rèn luyện kỹ năng nghiên cứu tài liệu, làm việc nhóm, quản lý tiến độ bằng Jira và quản lý mã nguồn bằng GitHub.

Mặc dù nhóm đã cố gắng hoàn thành đồ án một cách tốt nhất, tuy nhiên do thời gian thực hiện và kinh nghiệm thực tế còn hạn chế nên báo cáo chắc chắn vẫn còn nhiều thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ quý Thầy, Cô để có thể tiếp tục hoàn thiện sản phẩm trong tương lai.

Cuối cùng, nhóm xin kính chúc quý Thầy, Cô luôn dồi dào sức khỏe, hạnh phúc và thành công trong sự nghiệp giảng dạy.

Xin chân thành cảm ơn!

Chapter 1

GIỚI THIỆU

1.1 Giới thiệu đề tài

Trong bối cảnh trí tuệ nhân tạo (Artificial Intelligence - AI) phát triển mạnh mẽ, các mô hình ngôn ngữ lớn (Large Language Models - LLM) đang được ứng dụng rộng rãi trong nhiều lĩnh vực như giáo dục, y tế, tài chính và dịch vụ khách hàng. Đặc biệt, các hệ thống Chatbot thông minh đã trở thành công cụ hỗ trợ hiệu quả trong việc tìm kiếm, tổng hợp và trả lời thông tin.

Trong lĩnh vực giáo dục, sinh viên thường gặp khó khăn khi phải tìm kiếm thông tin từ nhiều nguồn tài liệu khác nhau như giáo trình, slide bài giảng, tài liệu PDF hoặc tài liệu tham khảo. Việc đọc và tổng hợp lượng lớn dữ liệu mất nhiều thời gian và đôi khi không mang lại kết quả chính xác.

Xuất phát từ nhu cầu đó, nhóm thực hiện đề tài:

XÂY DỰNG HỆ THỐNG CHATBOT HỖ TRỢ HỌC TẬP ỨNG DỤNG RETRIEVAL-AUGMENTED GENERATION (RAG)

Hệ thống được xây dựng với mục tiêu hỗ trợ sinh viên đặt câu hỏi trực tiếp về nội dung môn học. Chatbot sẽ truy xuất thông tin từ tài liệu đã được tải lên, kết hợp cùng mô hình ngôn ngữ lớn để tạo ra câu trả lời có độ chính xác cao và sát với tài liệu gốc.

Bên cạnh đó, đề tài còn tập trung nghiên cứu, đánh giá và tối ưu hóa quy trình RAG thông qua việc so sánh các chiến lược phân mảnh dữ liệu (Chunking Strategies) và các mô hình biểu diễn từ (Embedding Models) nhằm xác định giải pháp mang lại hiệu năng cao nhất cho bài toán hỏi đáp tài liệu học tập.

1.2 Lý do chọn đề tài

Hiện nay, phần lớn sinh viên sử dụng các công cụ AI phổ biến như ChatGPT hoặc Gemini để hỗ trợ học tập. Tuy nhiên, các mô hình này chủ yếu dựa trên dữ liệu được huấn luyện trước (pre-trained data) và không thể truy cập trực tiếp vào tài liệu nội bộ, giáo trình đặc thù của từng môn học nếu không được người dùng chủ động cung cấp một cách thủ công. Điều này dẫn đến một số hạn chế nhất định như: câu trả lời có thể không bám sát nội dung môn học, khó trích dẫn nguồn gốc tài liệu, khó cập nhật nhanh khi nội dung giáo trình thay đổi, và độ chính xác thường suy giảm đáng kể đối với các mảng kiến thức chuyên ngành sâu.

Trong khi đó, kỹ thuật Retrieval-Augmented Generation (RAG) nổi lên như một giải pháp đột phá, cho phép hệ thống tự động truy xuất trực tiếp các luồng thông tin từ các tài liệu do người dùng cung cấp, trước khi chuyển giao ngữ cảnh cho mô hình ngôn ngữ để sinh câu trả lời. Cơ chế này không chỉ giúp đảm bảo tính chính xác và khả năng trích dẫn minh bạch mà còn làm giảm thiểu đáng kể hiện tượng "ảo giác" (hallucination) thường gặp ở các LLM thông thường.

Nhận thấy tiềm năng to lớn này, nhóm quyết định lựa chọn nghiên cứu và xây dựng hệ thống Chatbot chuyên sâu về công nghệ RAG. Qua đề tài, nhóm kỳ vọng không chỉ tạo ra một công cụ hỗ trợ đắc lực cho sinh viên trong quá trình tự học, mà còn cung cấp một nền tảng Benchmark để phân tích và đánh giá định lượng các thành phần kỹ thuật lõi bên trong RAG.

1.3 Mục tiêu đề tài

Đề tài hướng đến việc giải quyết các bài toán về truy xuất và tổng hợp tri thức với những mục tiêu cụ thể như sau:

- Xây dựng hệ thống Chatbot hoàn chỉnh có khả năng hỗ trợ trả lời các câu hỏi chuyên sâu liên quan trực tiếp đến tài liệu môn học của sinh viên.
- Xây dựng quy trình xử lý tài liệu tự động (Ingestion Pipeline) bao gồm các bước: đọc file PDF/DOCX, làm sạch văn bản, phân mảnh dữ liệu (Chunking), tạo vector (Embedding) và lưu trữ hiệu quả trong cơ sở dữ liệu vector ChromaDB.
- Tích hợp mô hình ngôn ngữ lớn thông qua FastAPI để sinh câu trả lời tự nhiên, chính xác dựa trên ngữ cảnh được cung cấp.
- Xây dựng giao diện Web bằng ReactJS nhằm đem lại trải nghiệm tương tác mượt mà, thân thiện cho cả sinh viên và người quản trị.
- Thiết lập module Benchmark để tiến hành thử nghiệm, đo lường và so sánh hiệu năng giữa các cấu hình RAG khác nhau (ví dụ: Recursive vs Fixed-size Chunking, mô hình Embedding của Google vs BAAI/bge-m3) dựa trên bộ tiêu chí RAGAS.

1.4 Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu và triển khai giới hạn trong các nội dung sau: hệ thống hỗ trợ truy vấn thông tin chủ yếu từ tài liệu văn bản tĩnh (PDF, DOCX); triển khai kỹ thuật Retrieval-Augmented Generation (RAG) kết hợp với cơ sở dữ liệu vector ChromaDB; phát triển kiến trúc hệ thống Client-Server bao gồm Backend bằng FastAPI và Frontend bằng ReactJS; cuối cùng là việc đánh giá hiệu năng các mô hình thông qua bộ tiêu chuẩn đo lường định lượng.

Đề tài tạm thời chưa nghiên cứu các nội dung nâng cao như: nhận dạng giọng nói, xử lý hình ảnh phức tạp, hỗ trợ đa ngôn ngữ ở quy mô lớn hoặc việc tinh chỉnh mô hình sâu (Fine-Tuning) do những hạn chế về mặt tài nguyên phần cứng và thời gian thực hiện.

1.5 Đối tượng sử dụng

Hệ thống được thiết kế hướng tới hai nhóm người dùng chính yếu:

Sinh viên

Đây là nhóm đối tượng sử dụng trọng tâm của hệ thống. Sinh viên có thể trực tiếp đặt các câu hỏi về nội dung môn học và tra cứu thông tin nhanh chóng từ kho tài liệu. Bên cạnh đó, người dùng am hiểu kỹ thuật cũng có thể sử dụng tính năng thử nghiệm để tự cấu hình, so sánh kết quả sinh câu trả lời giữa các chiến lược RAG khác nhau, từ đó xem xét báo cáo đánh giá và tìm ra mô hình phù hợp nhất cho môn học của mình.

Quản trị viên

Quản trị viên đóng vai trò duy trì và mở rộng kho tri thức của hệ thống. Các quyền hạn chính bao gồm: tải lên và quản lý danh sách tài liệu môn học, theo dõi trạng thái phân mảnh và embedding của tài liệu, cũng như xem các báo cáo thống kê chung để đánh giá tổng quan về chất lượng hoạt động của hệ thống Chatbot.

1.6 Phương pháp nghiên cứu

Quá trình thực hiện đề tài được diễn ra tuần tự qua các giai đoạn:

1. Khảo sát thực trạng và các mô hình Chatbot hỗ trợ học tập hiện có trên thị trường.
2. Nghiên cứu chuyên sâu về kiến trúc Retrieval-Augmented Generation (RAG) và các kỹ thuật liên quan (Chunking, Vector Database, Prompt Engineering).

3. Thiết kế kiến trúc tổng thể cho hệ thống và phác thảo giao diện người dùng.
4. Lập trình và xây dựng Backend với FastAPI, tích hợp ChromaDB và các API của LLM.
5. Lập trình Frontend với ReactJS và kết nối hoàn chỉnh các luồng dữ liệu.
6. Xây dựng bộ test, tiến hành kiểm thử chức năng và chạy Benchmark đánh giá hiệu năng.
7. Tổng hợp số liệu, phân tích kết quả và hoàn thiện tài liệu báo cáo.

1.7 Cấu trúc báo cáo

Tài liệu báo cáo đồ án được cấu trúc thành các chương như sau:

- Chương 1: Lời cảm ơn.
- Chương 2: Giới thiệu (Tổng quan về đề tài, lý do, mục tiêu và phạm vi).
- Chương 3: Cơ sở lý thuyết và công nghệ sử dụng.
- Chương 4: Đặc tả yêu cầu phần mềm (SRS).
- Chương 5: Thiết kế hệ thống (Kiến trúc, biểu đồ lớp, cơ sở dữ liệu).
- Chương 6: Triển khai hệ thống (Luồng xử lý RAG và các module).
- Chương 7: Kiểm thử và đánh giá (Thử nghiệm Benchmark và phân tích số liệu).
- Chương 8: Quản lý dự án bằng Jira và GitHub.
- Chương 9: Kết luận và hướng phát triển.

Chapter 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1 Trí tuệ nhân tạo (Artificial Intelligence)

Trí tuệ nhân tạo (Artificial Intelligence - AI) là lĩnh vực nghiên cứu thuộc khoa học máy tính nhằm xây dựng các hệ thống có khả năng mô phỏng trí thông minh của con người. AI cho phép máy tính thực hiện nhiều nhiệm vụ như học tập, suy luận, lập kế hoạch, xử lý ngôn ngữ tự nhiên, nhận dạng hình ảnh và đưa ra quyết định.

Trong những năm gần đây, AI phát triển mạnh nhờ sự gia tăng về năng lực tính toán, dữ liệu lớn (Big Data) và các mô hình học sâu (Deep Learning). AI đã được ứng dụng rộng rãi trong nhiều lĩnh vực như giáo dục, tài chính, thương mại điện tử, y tế, giao thông và dịch vụ khách hàng. Trong đề tài này, AI đóng vai trò là nền tảng giúp hệ thống Chatbot hiểu câu hỏi của người dùng, truy xuất thông tin từ tài liệu và sinh ra câu trả lời phù hợp.

2.2 Mô hình ngôn ngữ lớn (Large Language Model)

Large Language Model (LLM) là các mô hình học sâu được huấn luyện trên tập dữ liệu văn bản rất lớn nhằm hiểu và sinh ngôn ngữ tự nhiên. Các LLM hiện đại sở hữu năng lực vượt trội trong việc thực hiện đa dạng các tác vụ xử lý ngôn ngữ như trả lời câu hỏi phức tạp, tóm tắt văn bản dài, dịch thuật đa ngôn ngữ, sinh mã nguồn lập trình, phân tích cấu trúc tài liệu, và duy trì ngữ cảnh hội thoại tự nhiên với người dùng.

Các mô hình tiêu biểu và phổ biến hiện nay bao gồm họ GPT của OpenAI, Llama của Meta, Gemini của Google, Claude của Anthropic, hay Qwen của Alibaba. Trong khuôn khổ dự án này, LLM (điển hình như Gemini) được tích hợp để đóng vai trò là bộ não xử lý cuối cùng, chịu trách nhiệm tổng hợp các luồng thông tin thô được cung cấp từ hệ thống truy xuất (Retrieval) và tạo ra

câu trả lời tự nhiên, chính xác, bám sát với ngữ cảnh mà người dùng yêu cầu.

2.3 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) là một mô hình kiến trúc tiên tiến kết hợp sức mạnh của hệ thống truy xuất thông tin (Information Retrieval) và khả năng sinh văn bản của mô hình ngôn ngữ lớn (Generation). Thay vì chỉ hoàn toàn phụ thuộc vào những kiến thức tĩnh đã được đóng gói sẵn trong trọng số của LLM (kiến thức tiền huấn luyện), RAG cho phép hệ thống chủ động tìm kiếm và trích xuất các đoạn tài liệu có liên quan trực tiếp từ một cơ sở dữ liệu động, trước khi gửi bối cảnh đó vào mô hình AI để xử lý.

Quy trình hoạt động cốt lõi của RAG được diễn ra tuần tự qua các bước: đầu tiên, người dùng gửi một câu hỏi hoặc yêu cầu hệ thống. Câu hỏi này ngay lập tức được mã hóa thành các Vector đa chiều (Vector Embedding) thông qua mô hình Embedding. Hệ thống sau đó tiến hành tìm kiếm và trích xuất các đoạn văn bản (chunks) có độ tương đồng ngữ nghĩa cao nhất từ cơ sở dữ liệu vector (như ChromaDB). Cuối cùng, các đoạn tài liệu này được chèn vào Prompt dưới dạng ngữ cảnh bổ sung và gửi cho LLM. Kết quả là LLM sẽ sinh ra câu trả lời dựa trên lượng thông tin vừa được cung cấp. Cơ chế này không chỉ giúp câu trả lời luôn bám sát tài liệu môn học, đảm bảo tính minh bạch thông qua việc trích dẫn nguồn, mà còn giải quyết triệt để hiện tượng "ảo giác" (Hallucination) — khi AI tự bịa ra thông tin sai lệch do thiếu kiến thức nền.

2.4 Embedding

Embedding là kỹ thuật cốt lõi trong xử lý ngôn ngữ tự nhiên, đóng vai trò chuyển đổi các đoạn văn bản (từ vựng, câu, hoặc đoạn văn) thành các vector số học có hàng trăm hoặc hàng nghìn chiều. Khác với các phương pháp tìm kiếm từ khóa truyền thống (như BM25 hay TF-IDF) chỉ dựa trên sự trùng khớp mặt chữ, các vector Embedding có khả năng phản ánh sâu sắc ý nghĩa ngữ nghĩa (semantics) và mối liên hệ ngữ cảnh của văn bản.

Quá trình Embedding trong hệ thống được thiết kế bao gồm việc đọc và bóc tách nội dung từ tài liệu PDF/DOCX, sau đó áp dụng các thuật toán chia nhỏ văn bản (Chunking) để tạo ra các đoạn thông tin có kích thước phù hợp. Tiếp theo, hệ thống sử dụng các mô hình Embedding (như Google Gemini Embedding hoặc BAAI/bge-m3) để sinh Vector Embedding cho từng đoạn, rồi lưu trữ toàn bộ vào ChromaDB. Khi người dùng đặt câu hỏi, hệ thống sẽ thực hiện biến đổi câu hỏi thành Vector tương ứng và tính toán độ đo khoảng cách (phổ biến nhất là Cosine Similarity) để xác định các đoạn văn bản có ý nghĩa gần nhất.

2.5 Cơ sở dữ liệu Vector (ChromaDB)

ChromaDB là một hệ quản trị cơ sở dữ liệu Vector mã nguồn mở, được thiết kế chuyên biệt để lưu trữ và truy vấn tốc độ cao đối với các dữ liệu Embedding. Khác với cơ sở dữ liệu quan hệ truyền thống (RDBMS) lưu trữ dữ liệu dưới dạng bảng và tìm kiếm bằng SQL, ChromaDB lưu trữ các Vector đa chiều đại diện cho ngữ nghĩa của văn bản.

Trong kiến trúc của hệ thống Chatbot, ChromaDB đóng vai trò như một bộ nhớ dài hạn, đảm nhận các tác vụ quan trọng như lưu trữ hàng triệu Vector sinh ra từ tài liệu, thực hiện tìm kiếm nội dung tương đồng (Similarity Search) với độ trễ thấp, và truy xuất nhanh chóng các đoạn văn bản nguyên gốc (metadata) để cung cấp cho mô hình AI. Việc ứng dụng ChromaDB không chỉ tối ưu tốc độ phản hồi của Chatbot mà còn mang lại khả năng mở rộng (scalability) linh hoạt khi số lượng tài liệu môn học gia tăng trong tương lai.

2.6 Backend Framework (FastAPI)

FastAPI là một Web Framework hiện đại, hiệu năng cao dùng để phát triển các ứng dụng RESTful API bằng ngôn ngữ lập trình Python. FastAPI nổi bật nhờ khả năng hỗ trợ lập trình bất đồng bộ (Asynchronous Programming) tối ưu hóa tốc độ xử lý I/O, tự động sinh tài liệu API (Swagger/Redoc) trực quan dựa trên chuẩn OpenAPI, và tính tương thích cao với các thư viện Machine Learning, AI vốn chủ yếu được viết bằng Python.

Trong kiến trúc tổng thể của đề tài, FastAPI đóng vai trò là xương sống (Backend) của toàn bộ hệ thống. FastAPI chịu trách nhiệm tiếp nhận và định tuyến các yêu cầu (Requests) từ phía giao diện Frontend, quản lý tiến trình tải lên và bóc tách tài liệu PDF, giao tiếp với ChromaDB để thực hiện các luồng truy xuất RAG, gọi API của các mô hình ngôn ngữ lớn (LLM), và điều phối các tác vụ tính toán chuyên sâu cho module Benchmark.

2.7 Frontend Framework (ReactJS)

ReactJS là thư viện JavaScript mã nguồn mở được phát triển bởi Meta, chuyên dùng để xây dựng các giao diện người dùng (User Interface) theo hướng Single Page Application (SPA). Ưu điểm cốt lõi của React nằm ở kiến trúc dựa trên Component (thành phần hóa), cho phép tái sử dụng code hiệu quả, cùng với cơ chế Virtual DOM giúp cập nhật và hiển thị dữ liệu theo thời gian thực một cách vô cùng mượt mà.

Đối với hệ thống này, ReactJS được ứng dụng để phát triển toàn bộ giao diện tương tác phía người dùng (Frontend). React giúp xây dựng màn hình Chat Sandbox trực quan, tích hợp các công cụ cấu hình linh hoạt cho mô hình AI, thiết kế các bảng điều khiển (Dashboard) quản lý tài liệu, và đặc biệt là vẽ các biểu đồ trực quan (như Radar Chart) để hiển thị kết quả đánh giá (Benchmark) và phân tích hiệu năng của các cấu hình RAG khác nhau.

2.8 Kiến trúc tổng thể hệ thống

Hình 2.1 mô tả kiến trúc tổng thể của hệ thống Chatbot hỗ trợ học tập, được thiết kế theo mô hình Client-Server.

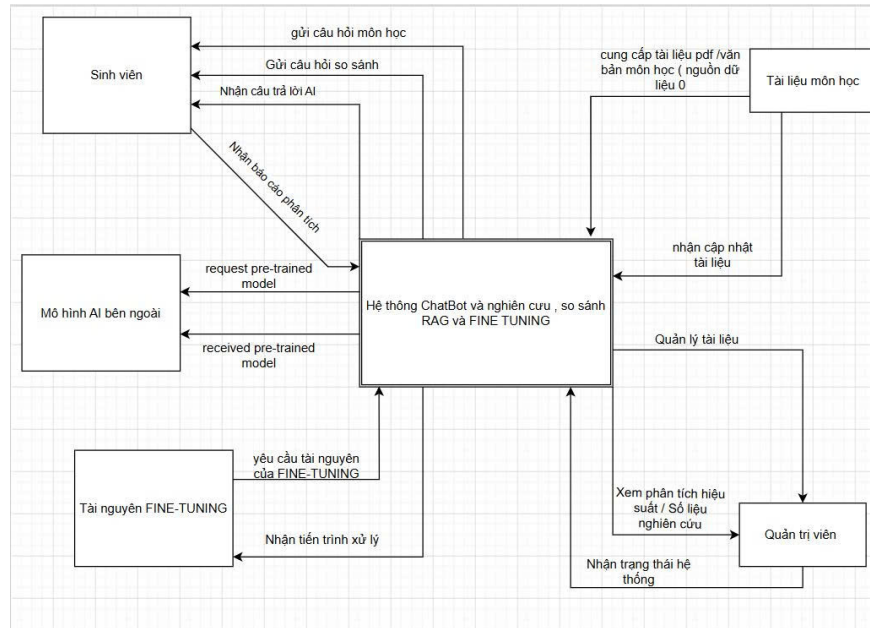


Figure 2.1: Kiến trúc tổng thể hệ thống

Kiến trúc hệ thống được chia làm năm thành phần chính: Người dùng (Sinh viên), Quản trị viên, Hệ thống Chatbot (Backend), Kho tài liệu môn học (Vector Database), và Mô hình AI (LLM & Embedding). Sinh viên tương tác với hệ thống qua giao diện Web để gửi câu hỏi; Backend sẽ tiếp nhận, sử dụng kỹ thuật RAG để trích xuất thông tin liên quan từ Kho tài liệu, kết hợp với Mô hình AI để sinh câu trả lời và phản hồi lại cho người dùng. Trong khi đó, Quản trị viên sử dụng hệ thống để cập nhật nguồn tài liệu mới, theo dõi trạng thái, và đánh giá hiệu suất thông qua công cụ Benchmark. Nhờ kiến trúc phân tách rõ ràng này, dữ liệu được quản lý tập trung, quy trình xử lý AI được tối ưu, giúp hệ thống dễ dàng bảo trì và mở rộng trong tương lai.

Chapter 3

ĐẶC TẢ YÊU CẦU PHẦN MỀM (SRS)

3.1 Giới thiệu

Tài liệu Đặc tả yêu cầu phần mềm (Software Requirements Specification - SRS) mô tả đầy đủ và chi tiết các yêu cầu đối với hệ thống Chatbot hỗ trợ học tập ứng dụng công nghệ Retrieval-Augmented Generation (RAG). Tài liệu này đóng vai trò là cơ sở cốt lõi để nhóm phát triển tiến hành thiết kế kiến trúc, lập trình, kiểm thử và đánh giá mức độ hoàn thiện của hệ thống ở các giai đoạn sau.

Các yêu cầu được xây dựng dựa trên nhu cầu thực tế của sinh viên trong việc tra cứu, tổng hợp tài liệu học tập một cách nhanh chóng và chính xác. Đồng thời, hệ thống cũng đáp ứng mục tiêu nghiên cứu thông qua việc cung cấp công cụ Benchmark để đánh giá, so sánh hiệu quả giữa các chiến lược phân mảnh dữ liệu (Chunking) và các mô hình biểu diễn từ (Embedding Models) khác nhau.

3.2 Mục đích

Mục đích chính của tài liệu SRS bao gồm: xác định đầy đủ và chính xác các chức năng mà hệ thống Chatbot cần cung cấp cho từng nhóm người dùng; định nghĩa rõ ràng các yêu cầu phi chức năng (như hiệu năng, bảo mật, khả năng mở rộng); làm cơ sở vững chắc cho giai đoạn thiết kế cơ sở dữ liệu và giao diện; hỗ trợ quá trình kiểm thử và nghiệm thu sản phẩm cuối cùng; đồng thời giúp giảm thiểu tối đa sự sai lệch giữa kỳ vọng ban đầu và quá trình phát triển thực tế.

3.3 Phạm vi hệ thống

Hệ thống Chatbot được xây dựng nhằm mục đích chính là hỗ trợ sinh viên đặt câu hỏi liên quan đến nội dung môn học và nhận lại các câu trả lời chính xác, được trích xuất trực tiếp từ nguồn tài liệu tham khảo đáng tin cậy. Để thực hiện điều này, hệ thống tập trung triển khai chuyên sâu mô hình Retrieval-Augmented Generation (RAG).

Bên cạnh chức năng hỏi đáp, hệ thống còn cung cấp môi trường Sandbox và Benchmark cho phép người dùng kỹ thuật hoặc quản trị viên có thể: thử nghiệm các cấu hình RAG khác nhau (thay đổi Chunking, thay đổi Embedding Model, thay đổi LLM), quản lý hệ thống tài liệu động, theo dõi các chỉ số đánh giá (như Accuracy, Relevancy, Faithfulness theo chuẩn RAGAS) và xuất báo cáo thống kê trực quan.

3.4 Các tác nhân của hệ thống

Hệ thống được thiết kế để tương tác với các tác nhân (Actors) chính bao gồm:

- Sinh viên (Người dùng cuối).
- Quản trị viên (Admin).
- Mô hình AI (Gemini, Open Source LLMs).
- Kho tài liệu môn học (Vector Database).

3.5 Biểu đồ ngữ cảnh (Context Diagram)

Biểu đồ Context mô tả hệ thống ở mức độ trừu tượng cao nhất, thể hiện rõ ràng các tác nhân bên ngoài và các luồng dữ liệu chính được trao đổi với hệ thống Chatbot.

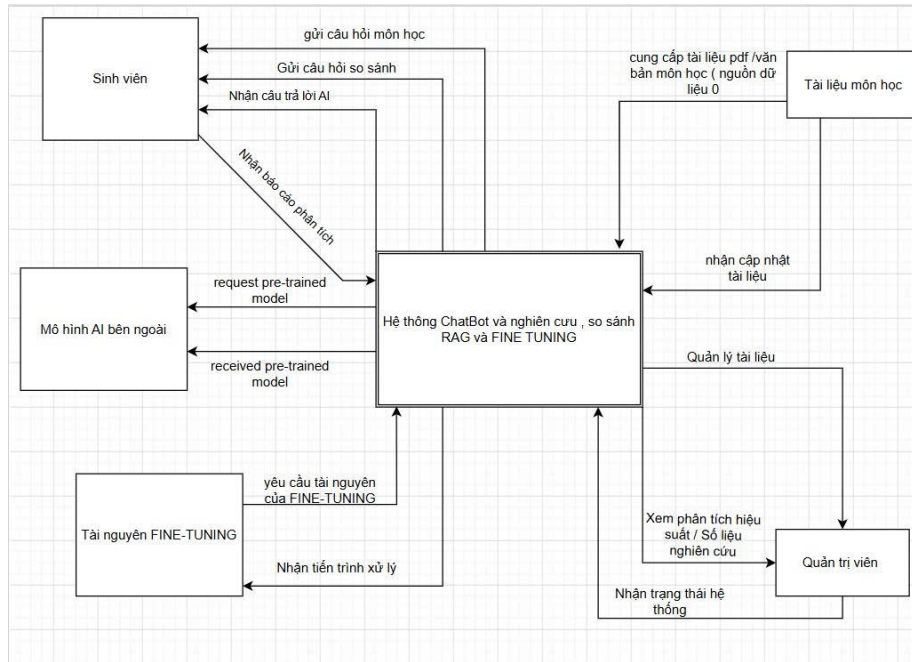


Figure 3.1: Biểu đồ Context của hệ thống

Trong biểu đồ Context, hệ thống Chatbot (Backend FastAPI) đóng vai trò trung tâm tiếp nhận, điều phối và xử lý mọi yêu cầu từ các tác nhân bên ngoài. Sinh viên gửi câu hỏi về môn học, hệ thống tiếp nhận và trả về câu trả lời tự nhiên kèm theo nguồn tài liệu trích dẫn rõ ràng. Kho tài liệu cung cấp dữ liệu thô (PDF, DOCX), sau đó được hệ thống xử lý (Chunking, Embedding) để phục vụ quá trình tìm kiếm. Mô hình AI (như Gemini 2.5 Flash) đóng vai trò bên thứ ba, nhận Prompt đã được bơm ngữ cảnh từ hệ thống và sinh ra câu trả lời cuối cùng. Quản trị viên tương tác với hệ thống để quản lý dữ liệu, theo dõi trạng thái máy chủ và xem xét các biểu đồ phân tích hiệu suất (Dashboard).

3.6 Biểu đồ Use Case

Biểu đồ Use Case mô tả chi tiết toàn bộ các chức năng cụ thể mà hệ thống cung cấp, được phân chia theo quyền hạn của từng tác nhân.

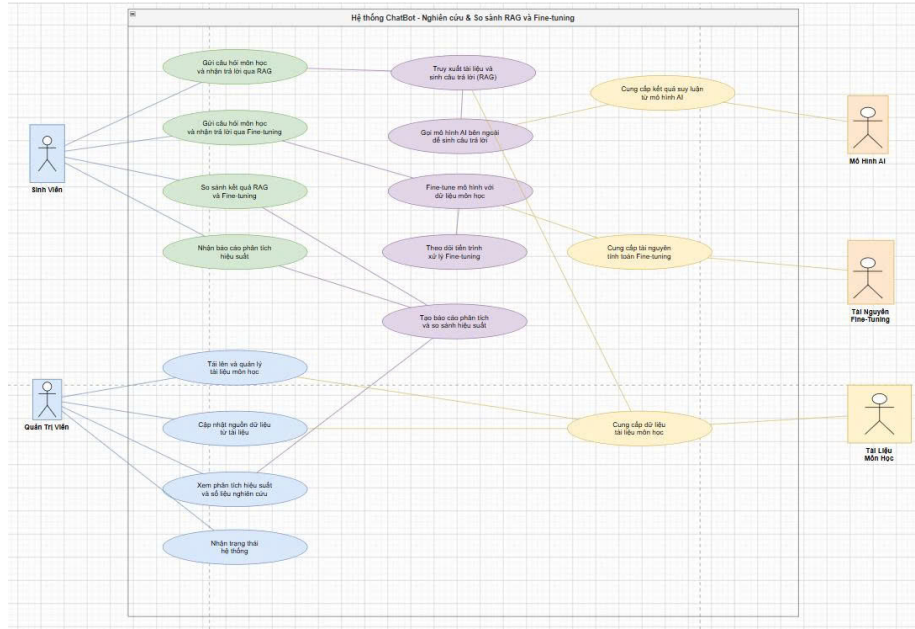


Figure 3.2: Biểu đồ Use Case

Biểu đồ Use Case cho thấy hệ thống phân tách rõ rệt quyền hạn giữa hai nhóm người dùng chính là Sinh viên và Quản trị viên.

Đối với Sinh viên, các chức năng chính bao gồm: gửi câu hỏi để Chatbot tư vấn dựa trên cơ chế RAG; xem lại lịch sử hội thoại; thử nghiệm thay đổi các thông số cấu hình RAG (Top-K, Chunking Strategy, Embedding Model) ngay trong giao diện Chat Sandbox; và xem báo cáo đánh giá chất lượng câu trả lời.

Đối với Quản trị viên, quyền hạn được mở rộng bao gồm: tải lên hàng loạt tài liệu học tập mới; quản lý, cập nhật hoặc xóa tài liệu trong kho tri thức ChromaDB; theo dõi trạng thái xử lý (Ingestion) của tài liệu; quản lý danh sách người dùng; thực thi các kịch bản Benchmark quy mô lớn; và phân tích dữ liệu hiệu năng thông qua Dashboard thống kê tổng quan.

3.7 Yêu cầu chức năng

Để đáp ứng các quy trình nghiệp vụ đã đề ra, hệ thống cần hoàn thiện các yêu cầu chức năng sau:

1. Cho phép người dùng nhập câu hỏi văn bản thông qua giao diện Chat.
2. Tự động trích xuất các đoạn văn bản (chunks) liên quan nhất từ ChromaDB dựa trên thuật toán tìm kiếm vector.
3. Sinh câu trả lời thông minh, tự nhiên bằng công nghệ RAG, đồng thời hiển thị danh sách tài liệu trích dẫn tương ứng.

4. Lưu trữ và hiển thị lịch sử hội thoại, cho phép người dùng tiếp tục ngữ cảnh trò chuyện.
5. Cung cấp tính năng Upload tài liệu PDF/DOCX, tự động phân mảnh và tạo Vector Embedding.
6. Cung cấp module Benchmark cho phép chạy thử nghiệm với nhiều cấu hình Chunking và Embedding khác nhau.
7. Tự động tính toán các chỉ số đánh giá (Accuracy, Relevancy, Latency, v.v.) và xuất báo cáo Radar Chart.
8. Cho phép quản trị viên quản lý danh sách tài liệu và trạng thái hoạt động của hệ thống.

3.8 Yêu cầu phi chức năng

Bên cạnh chức năng, hệ thống cần đảm bảo các tiêu chuẩn về chất lượng (Phi chức năng) như sau:

1. **Hiệu năng (Performance):** Thời gian phản hồi (Latency) đối với một câu hỏi thông thường cần được duy trì dưới 5 giây. Hệ thống phải đảm bảo truy xuất ChromaDB và gọi API của LLM trong thời gian tối ưu nhất.
2. **Độ chính xác (Accuracy & Faithfulness):** Hệ thống bắt buộc phải ưu tiên sử dụng tri thức từ tài liệu môn học để trả lời. Nếu thông tin không có trong tài liệu, Chatbot cần thông báo thay vì tự bịa ra thông tin sai lệch (Hallucination).
3. **Khả năng mở rộng (Scalability):** Hệ thống phải được thiết kế rời rạc (Decoupled), cho phép dễ dàng thêm mới các loại mô hình Embedding hoặc thay đổi LLM (ví dụ chuyển từ Gemini sang GPT) mà không làm gián đoạn mã nguồn hiện tại.
4. **Bảo mật (Security):** Cần có cơ chế phân quyền rõ ràng, chỉ quản trị viên mới được phép thao tác xóa, cập nhật tài liệu và can thiệp vào dữ liệu Benchmark cốt lõi của hệ thống.
5. **Khả dụng (Availability):** Kiến trúc Backend FastAPI phải đảm bảo khả năng xử lý bất đồng bộ (Asynchronous) tốt để phục vụ nhiều sinh viên tra cứu cùng lúc trong các đợt ôn thi cao điểm.

3.9 Kết luận chương

Chương này đã làm rõ đặc tả yêu cầu phần mềm của hệ thống Chatbot hỗ trợ học tập dựa trên RAG. Bằng việc phân tích mục đích, xác định rõ các tác nhân, xây dựng sơ đồ Context, sơ đồ Use Case và liệt kê các yêu cầu chức/phi chức

năng, nhóm đã có một định hướng kỹ thuật nhất quán. Đây chính là tiền đề quan trọng nhất để tiến hành thiết kế kiến trúc hệ thống chi tiết ở chương tiếp theo.

Chapter 4

THIẾT KẾ HỆ THỐNG

4.1 Giới thiệu

Sau khi xác định đầy đủ các yêu cầu trong tài liệu SRS, chương này sẽ trình bày toàn bộ quá trình thiết kế hệ thống Chatbot hỗ trợ học tập ứng dụng mô hình Retrieval-Augmented Generation (RAG). Nội dung cốt lõi bao gồm thiết kế kiến trúc phần mềm, xây dựng biểu đồ lớp (Class Diagram), thiết kế mô hình thực thể liên kết (ERD) cho cơ sở dữ liệu và đặc tả luồng dữ liệu (Data Flow) giữa các thành phần.

Việc thiết kế hệ thống được định hướng theo tiêu chí đảm bảo khả năng mở rộng linh hoạt, dễ dàng bảo trì và tối ưu cho việc tích hợp thêm các mô hình ngôn ngữ hoặc kỹ thuật Embedding mới. Đồng thời, kiến trúc phải hỗ trợ tốt cho module Benchmark để phục vụ mục tiêu đánh giá và so sánh các cấu hình RAG khác nhau.

4.2 Kiến trúc tổng thể

Hệ thống được xây dựng theo mô hình Client-Server hiện đại, kết hợp chặt chẽ với kiến trúc AI Pipeline chuyên biệt cho RAG. Các thành phần chính cấu thành nên hệ thống bao gồm:

- **Frontend ReactJS:** Chịu trách nhiệm render giao diện tương tác, biểu đồ Dashboard và xử lý logic phía người dùng.
- **Backend FastAPI:** Đóng vai trò là Controller trung tâm, nhận request, điều phối các tác vụ AI và giao tiếp với các Database.
- **ChromaDB:** Cơ sở dữ liệu Vector chuyên dụng để lưu trữ và truy xuất các Embedding sinh ra từ tài liệu.
- **LLM (Gemini/OpenAI):** Đóng vai trò sinh ngôn ngữ, nhận Prompt từ hệ thống và tạo câu trả lời cuối cùng.

- **Module Benchmark:** Thực thi các kịch bản kiểm thử, tính toán điểm số (Accuracy, Relevancy) và lưu trữ kết quả cấu hình tối ưu.
- **Cơ sở dữ liệu MySQL:** Lưu trữ siêu dữ liệu (metadata), thông tin người dùng, lịch sử đoạn chat và các báo cáo phân tích.

4.3 Thiết kế lớp (Class Diagram)

Biểu đồ lớp (Class Diagram) mô tả cấu trúc hướng đối tượng của toàn bộ Backend hệ thống, thể hiện các đối tượng tham gia vào quy trình xử lý và mối quan hệ giữa chúng.

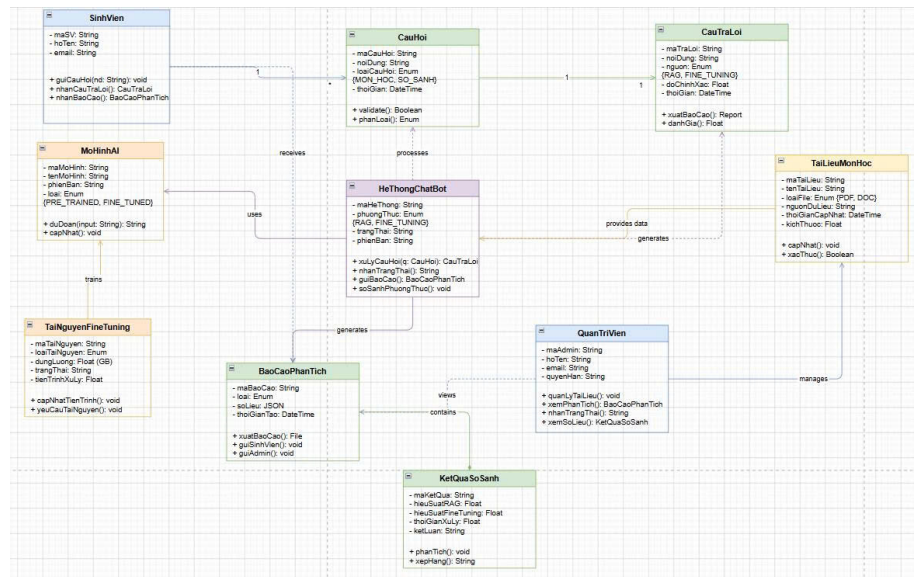


Figure 4.1: Biểu đồ lớp của hệ thống

Hệ thống được thiết kế xung quanh các lớp chính yếu sau:

SinhVien & QuanTriVien

Hai lớp này kế thừa từ một lớp User trừu tượng chung, đại diện cho những người sử dụng hệ thống. Lớp ‘SinhVien’ cung cấp các phương thức để gửi câu hỏi, nhận câu trả lời và tra cứu lịch sử hội thoại. Lớp ‘QuanTriVien’ được cấp quyền cao hơn, bổ sung các phương thức để quản lý danh sách tài liệu môn học, theo dõi biểu đồ thống kê hệ thống và khởi chạy các kịch bản kiểm thử quy mô lớn.

HeThongChatbot

Đây là lớp điều phối trung tâm của toàn bộ ứng dụng (Facade/Controller). Lớp này nhận đầu vào là câu hỏi từ 'CauHoi', sau đó tự động điều hướng luồng xử lý: gọi module xử lý văn bản, kết nối đến 'MoHinhAI' để tạo Embedding, truy vấn 'TaiLieuMonHoc' (thông qua ChromaDB), và cuối cùng là sinh Prompt để gọi LLM trả về kết quả.

TaiLieuMonHoc

Lớp này đóng gói các thuộc tính và hành vi liên quan đến một tài liệu tải lên (PDF, DOCX). Nó đảm nhiệm việc đọc luồng dữ liệu file, thực hiện chia nhỏ văn bản (Chunking) theo các chiến lược được thiết lập sẵn, và giao tiếp với cơ sở dữ liệu Vector để lưu trữ các đoạn text đã được phân mảnh.

MoHinhAI

Đây là một Interface trừu tượng đại diện cho các dịch vụ AI bên ngoài (như Gemini, GPT, hoặc các mô hình Embedding mã nguồn mở). Lớp này quy định chuẩn giao tiếp chung cho việc gửi Prompt và nhận Text, hoặc gửi Text và nhận Vector, giúp hệ thống dễ dàng thay thế hoặc nâng cấp mô hình AI trong tương lai mà không cần sửa đổi các lớp khác.

KetQuaSoSanh & BaoCaoPhanTich

Các lớp này thuộc về module Benchmark. 'KetQuaSoSanh' thực hiện tính toán độ đo (Metrics) giữa các cấu hình Chunking và Embedding khác nhau dựa trên tập test set. 'BaoCaoPhanTich' sau đó sẽ tổng hợp các số liệu này (Accuracy, Latency, Relevancy) để kết xuất thành báo cáo và cung cấp dữ liệu cho việc vẽ biểu đồ Radar trên Frontend.

4.4 Thiết kế cơ sở dữ liệu

Hệ thống kết hợp sử dụng cơ sở dữ liệu quan hệ (MySQL) cho dữ liệu có cấu trúc và cơ sở dữ liệu Vector (ChromaDB) cho dữ liệu Embedding. Biểu đồ ERD dưới đây mô tả cấu trúc của hệ cơ sở dữ liệu quan hệ.

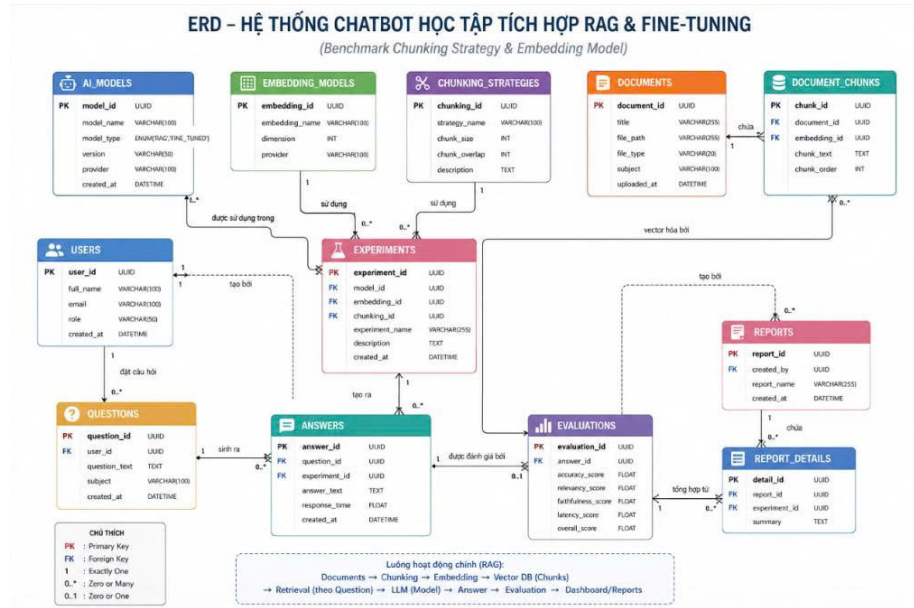


Figure 4.2: ERD của hệ thống

Các bảng dữ liệu cốt lõi bao gồm:

Users, Questions, Answers

Bảng ‘Users’ lưu trữ thông tin xác thực và phân quyền của sinh viên và quản trị viên. Bảng ‘Questions’ lưu lại lịch sử câu hỏi do người dùng gửi lên, liên kết 1-n với bảng ‘Users’. Bảng ‘Answers’ lưu trữ câu trả lời tương ứng được AI sinh ra, liên kết 1-1 với ‘Questions’. Cấu trúc này cho phép khôi phục toàn bộ bối cảnh của một phiên trò chuyện (Session).

Documents & Document Chunks

Bảng ‘Documents’ quản lý metadata của tài liệu gốc (tên file, đường dẫn lưu trữ, ngày tải lên, môn học). Bảng ‘Document Chunks’ (được đồng bộ logic với ChromaDB) lưu thông tin về các đoạn văn bản đã bị cắt nhỏ từ ‘Documents’. Bảng này giúp hệ thống đối chiếu và truy xuất ngược lại vị trí gốc của đoạn văn (ví dụ: nằm ở trang số mấy của file PDF nào) để tạo chức năng trích dẫn nguồn cho Chatbot.

Chunking Strategies & Embedding Models

Đây là các bảng cấu hình tham số. ‘Chunking Strategies’ lưu trữ các quy tắc chia văn bản (ví dụ: Recursive, Fixed-size, Semantic). ‘Embedding Models’ lưu

trữ danh sách các mô hình tạo vector được hệ thống hỗ trợ. Hai bảng này là cơ sở để người dùng lựa chọn cấu hình trong giao diện Chat Sandbox.

Experiments, Evaluations & Reports

Cụm bảng này phục vụ cho tính năng Benchmark. ‘Experiments’ đại diện cho một lần chạy thử nghiệm với một cấu hình (Chunking + Embedding) cụ thể. ‘Evaluations’ lưu điểm số chi tiết của từng tiêu chí (Accuracy, Faithfulness, Latency) cho từng câu hỏi trong tập test. Cuối cùng, ‘Reports’ sẽ lưu kết quả thống kê tổng hợp (Average Score) của toàn bộ đợt thử nghiệm đó để hiển thị lên Dashboard.

4.5 Luồng xử lý dữ liệu (RAG Pipeline)

Mọi truy vấn của người dùng đều trải qua một quy trình xử lý khép kín và tự động. Dưới đây là mô tả chi tiết luồng dữ liệu chuẩn (Standard RAG Pipeline) của hệ thống:

1. **Tiếp nhận:** Frontend ReactJS gửi câu hỏi của sinh viên kèm theo cấu hình RAG hiện tại (nếu có) thông qua REST API tới Backend FastAPI.
2. **Tạo Vector câu hỏi:** Backend sử dụng ‘MoHinhAI’ (được chỉ định trong cấu hình) để chuyển đổi nội dung câu hỏi thành một Vector đa chiều.
3. **Truy xuất (Retrieval):** Backend kết nối tới ChromaDB, thực thi thuật toán Cosine Similarity để so sánh vector câu hỏi với hàng ngàn vector tài liệu có sẵn, từ đó rút trích ra Top-K đoạn văn bản (chunks) có ý nghĩa gần gũi nhất.
4. **Xây dựng Prompt:** Các đoạn văn bản Top-K này được Backend tổng hợp, làm sạch và chèn vào một Template Prompt chuẩn, yêu cầu AI chỉ được phép trả lời dựa trên lượng thông tin này.
5. **Sinh kết quả (Generation):** Prompt hoàn chỉnh được gửi qua API tới mô hình LLM (ví dụ Gemini 2.5 Flash). LLM suy luận và sinh ra câu trả lời cuối cùng.
6. **Phản hồi:** Backend ghi nhận lịch sử vào cơ sở dữ liệu MySQL, tính toán thời gian phản hồi (Latency), gán thông tin nguồn trích dẫn và trả về cho ReactJS để render ra giao diện.

4.6 Kết luận chương

Chương này đã phác thảo bức tranh kỹ thuật toàn diện của hệ thống thông qua việc trình bày kiến trúc Client-Server kết hợp AI Pipeline, mô hình hướng đối tượng Class Diagram và cơ sở dữ liệu quan hệ ERD. Việc thiết kế lấy RAG và Benchmark làm trung tâm không chỉ đảm bảo ứng dụng chạy mượt mà, mà còn

tạo nền tảng vững chắc để đo lường, tinh chỉnh và tối ưu hóa hệ thống ở giai đoạn triển khai kế tiếp.

Chapter 5

TRIỂN KHAI HỆ THỐNG

5.1 Môi trường triển khai

Hệ thống Chatbot được triển khai thực tế theo mô hình Client-Server nhằm đảm bảo sự phân tách rõ ràng giữa giao diện hiển thị và logic xử lý nghiệp vụ. Phía Client (Frontend) được xây dựng bằng thư viện ReactJS kết hợp với Tailwind CSS, mang lại giao diện người dùng hiện đại, tối giản và đáp ứng nhanh (responsive). Phía Server (Backend) sử dụng framework FastAPI của Python, cho phép xử lý các luồng dữ liệu bất đồng bộ (Asynchronous) hiệu quả, đặc biệt là khi phải thực hiện các tác vụ nặng như tạo Vector Embedding hay gọi API tới các mô hình ngôn ngữ. Toàn bộ dữ liệu Vector sinh ra từ tài liệu môn học được lưu trữ tập trung trên ChromaDB, đảm bảo tốc độ truy vấn tính bằng mili-giây. Lõi xử lý ngôn ngữ của hệ thống tận dụng sức mạnh của mô hình Gemini 2.5 Flash thông qua API, kết hợp chặt chẽ với cơ chế Retrieval-Augmented Generation (RAG) để đảm bảo câu trả lời luôn dựa trên tri thức nội bộ.

Về mặt tổ chức, mã nguồn được chia thành các module độc lập bao gồm: Giao diện người dùng (Frontend Sandbox), API xử lý nghiệp vụ (FastAPI Routers), Module RAG (xử lý Chunking & Embedding), Vector Database (ChromaDB), Module Benchmark (chạy thử nghiệm tự động) và Dashboard thống kê (tổng hợp điểm số).

5.2 Chức năng quản lý tài liệu

Một trong những tiền đề quan trọng nhất của hệ thống RAG là việc xây dựng một kho tri thức phong phú và chất lượng. Chức năng quản lý tài liệu cho phép quản trị viên tải lên (Upload) các giáo trình, slide bài giảng định dạng PDF hoặc DOCX.

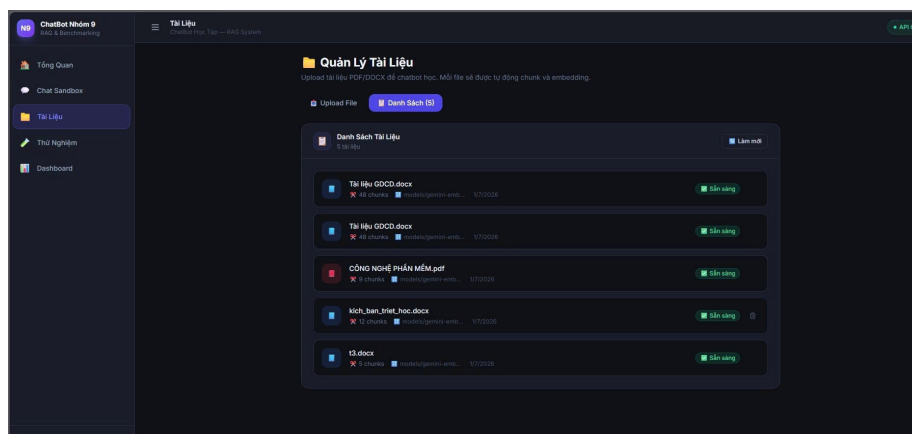


Figure 5.1: Chức năng quản lý tài liệu

Ngay khi tài liệu được tải lên hệ thống, Backend sẽ tự động kích hoạt một luồng xử lý văn bản (Ingestion Pipeline) phức tạp. Ban đầu, thư viện PyPDF hoặc pdfplumber sẽ quét qua toàn bộ tài liệu để trích xuất văn bản thô (raw text), loại bỏ các ký tự nhiễu và chuẩn hóa định dạng. Sau đó, văn bản thô này sẽ đi qua quá trình phân mảnh (Chunking). Hệ thống hỗ trợ nhiều chiến lược phân mảnh khác nhau (như Recursive Character Text Splitter) nhằm cắt tài liệu thành các khối (chunk) có độ dài tối ưu (thường từ 500 đến 1000 ký tự) mà vẫn giữ nguyên được ngữ cảnh của đoạn văn.

Tiếp theo, hệ thống sử dụng một mô hình Embedding (ví dụ: Google Gemini Embedding hoặc BAAI/bge-m3) để biến đổi các đoạn văn bản này thành các Vector đa chiều. Cuối cùng, các Vector này cùng với siêu dữ liệu (metadata như: tên file, số trang) được lưu trữ vào ChromaDB. Trạng thái của tài liệu sẽ được cập nhật thành "Sẵn sàng", lúc này người dùng có thể bắt đầu đặt câu hỏi liên quan đến nội dung tài liệu vừa tải lên.

5.3 Chức năng Chat Sandbox

Chat Sandbox là không gian làm việc chính của hệ thống, nơi sinh viên trực tiếp tương tác với AI thông qua việc đặt câu hỏi bằng ngôn ngữ tự nhiên. Giao diện này không chỉ đóng vai trò là một hộp thoại thông thường, mà còn là một môi trường thử nghiệm (Sandbox) cho phép người dùng tùy chỉnh các thông số kỹ thuật của RAG ngay trong thời gian thực.

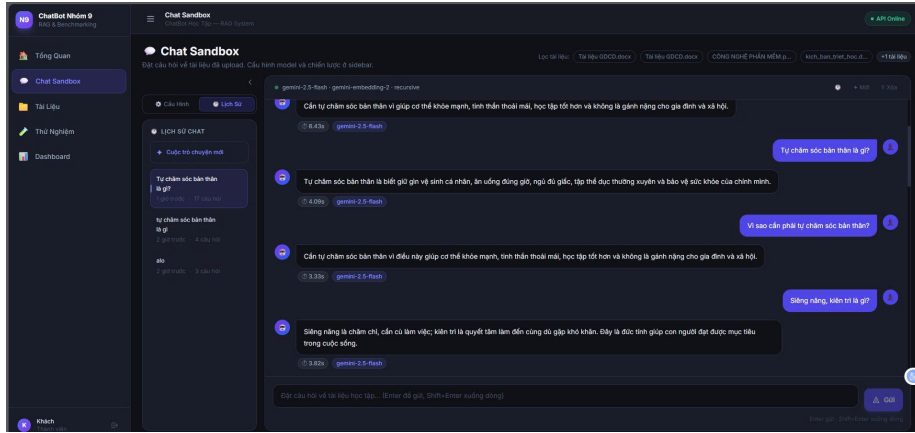


Figure 5.2: Giao diện Chat Sandbox

Giao diện hiển thị trực quan các thông số quan trọng như: mô hình AI đang được sử dụng (ví dụ: Gemini 2.5 Flash), thời gian trễ của API (Latency), cùng danh sách các tài liệu đang được chọn làm ngữ cảnh. Đặc biệt, người dùng có thể thiết lập hệ số Top-K (số lượng đoạn tài liệu liên quan tối đa được lấy ra từ database) hoặc thay đổi thuật toán Chunking ngay tại Sidebar. Chức năng lưu giữ ngữ cảnh hội thoại cũng được triển khai bằng cách truyền lại lịch sử tin nhắn trong các lần gọi API tiếp theo, giúp cuộc trò chuyện diễn ra tự nhiên và liền mạch mà không yêu cầu người dùng phải nhắc lại câu hỏi cũ.

5.4 Luồng xử lý kỹ thuật của hệ thống RAG

Để đảm bảo tốc độ và độ chính xác, luồng xử lý RAG từ lúc nhận câu hỏi đến khi trả về đáp án được thiết kế vô cùng chặt chẽ.

Đầu tiên, câu hỏi do sinh viên nhập vào sẽ được gửi qua REST API đến FastAPI. Tại đây, câu hỏi lập tức được đưa qua mô hình Embedding để sinh ra một Vector truy vấn (Query Vector). Vector này sau đó được chuyển cho ChromaDB để thực hiện thuật toán tìm kiếm khoảng cách tương đồng (như Cosine Similarity hoặc L2 Distance). ChromaDB sẽ duyệt qua không gian vector và trả về Top-K đoạn tài liệu (chunks) có ý nghĩa sát với câu hỏi nhất, bao gồm cả nội dung text và nguồn trích dẫn.

Ở bước tiếp theo, Backend tiến hành ghép nối (Augment) các đoạn tài liệu này vào một khuôn mẫu Prompt (Prompt Template) định sẵn. Mẫu Prompt này thường mang nội dung chỉ thị nghiêm ngặt, yêu cầu mô hình LLM (như Gemini) chỉ được phép sử dụng tri thức từ phần tài liệu đính kèm để trả lời, và nếu thông tin không có trong tài liệu, AI phải thông báo rõ ràng thay vì tự bịa đặt. Cuối cùng, Prompt hoàn chỉnh được gọi qua API tới LLM. LLM suy luận, tổng hợp thông tin, sinh ra câu trả lời và trả về giao diện Frontend, hoàn tất chu trình hỏi đáp một cách minh bạch và đáng tin cậy.

5.5 Kết luận chương

Chương này đã làm rõ quá trình triển khai hệ thống trong thực tế. Bằng việc phân tích sâu các luồng kỹ thuật từ quản lý tài liệu (Ingestion Pipeline) đến quá trình xử lý câu hỏi (Retrieval và Generation), có thể thấy hệ thống Chatbot được xây dựng không chỉ chú trọng về mặt giao diện mà còn tập trung giải quyết triệt để các bài toán tối ưu về tốc độ và tính toàn vẹn của dữ liệu trong mô hình RAG. Đây là cơ sở quan trọng để tiếp tục tiến hành các thử nghiệm Benchmark ở chương tiếp theo.

Chapter 6

KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

6.1 Mục tiêu và phương pháp kiểm thử

Mục tiêu của giai đoạn kiểm thử không chỉ dừng lại ở việc đảm bảo các chức năng phần mềm hoạt động đúng theo tài liệu đặc tả SRS (như chức năng Upload, Chat Sandbox), mà quan trọng hơn là đánh giá định lượng chất lượng của hệ thống trí tuệ nhân tạo. Để thực hiện điều này, nhóm đã sử dụng phương pháp kiểm thử tự động thông qua module Benchmark, kết hợp với bộ tiêu chuẩn RAGAS (Retrieval Augmented Generation Assessment) nhằm đo lường hiệu suất của hệ thống RAG dưới nhiều cấu hình khác nhau.

Các chỉ số (Metrics) chính được sử dụng để đánh giá bao gồm:

- **Accuracy (Độ chính xác):** Đo lường tỷ lệ câu trả lời hoàn toàn đúng so với đáp án chuẩn (Ground Truth).
- **Relevancy (Độ bám sát):** Đánh giá xem câu trả lời có thực sự giải quyết được trọng tâm câu hỏi hay không, tránh tình trạng trả lời lan man.
- **Faithfulness (Độ trung thực):** Chỉ số quan trọng nhất trong RAG, đo lường tỷ lệ thông tin trong câu trả lời được trích xuất hoàn toàn từ tài liệu nguồn (không bịa đặt thông tin).
- **Latency (Độ trễ):** Thời gian trung bình để hệ thống xử lý toàn bộ luồng RAG và sinh ra kết quả cuối cùng.

6.2 Môi trường thử nghiệm Benchmark

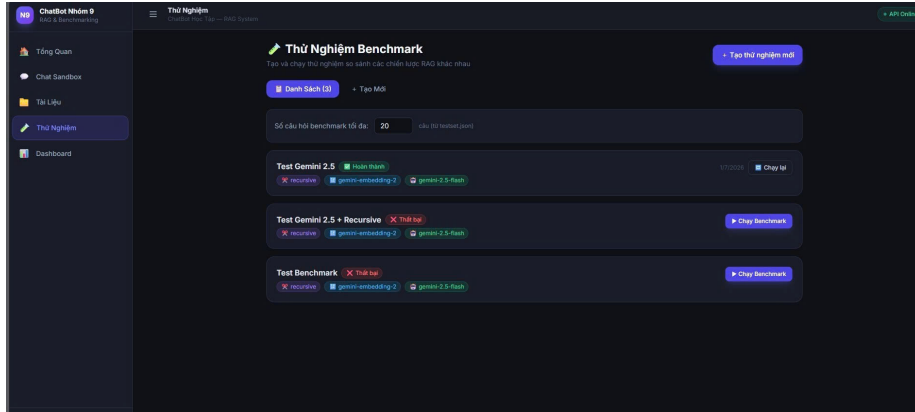


Figure 6.1: Giao diện cấu hình và chạy Benchmark

Để đảm bảo tính khách quan, nhóm đã xây dựng một tập dữ liệu kiểm thử (Test dataset) gồm 50 câu hỏi đa dạng (từ mức độ nhận biết định nghĩa đến suy luận phức tạp) được trích xuất trực tiếp từ các chương của giáo trình.

Trong module Benchmark, nhóm tiến hành chạy vòng lặp thử nghiệm đối chiếu giữa các cấu hình khác nhau, bao gồm:

- Thay đổi Chunking Strategy: Recursive Character Splitter so sánh với Fixed-size Splitter (đều cấu hình kích thước 1000 ký tự, độ trùm 200).
- Thay đổi Embedding Model: Mô hình thương mại Google Gemini Embedding so sánh với mô hình mã nguồn mở BAAI/bge-m3.

Mô hình LLM được cố định là Gemini 2.5 Flash để đảm bảo sự công bằng khi so sánh chất lượng thông tin truy xuất.

6.3 Kết quả và Phân tích số liệu

Sau khi hệ thống hoàn tất hàng loạt các kịch bản kiểm thử, toàn bộ kết quả được tự động tổng hợp, tính toán điểm số trung bình và kết xuất lên Dashboard trực quan.

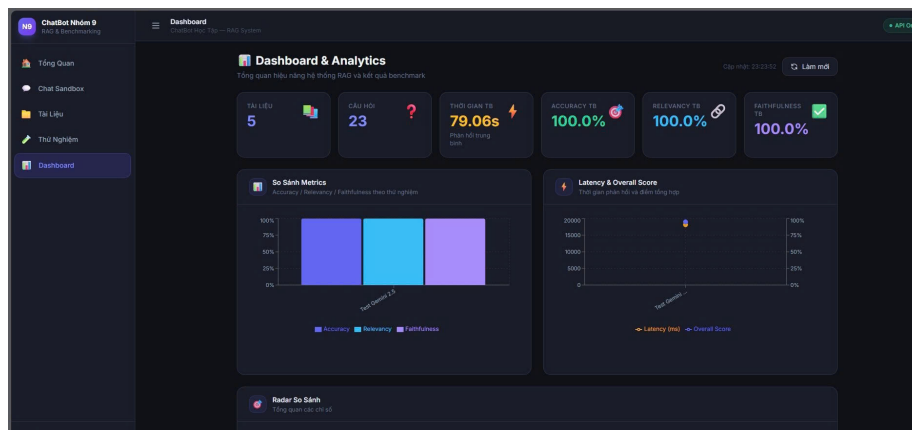


Figure 6.2: Dashboard tổng quan hiển thị các chỉ số RAGAS

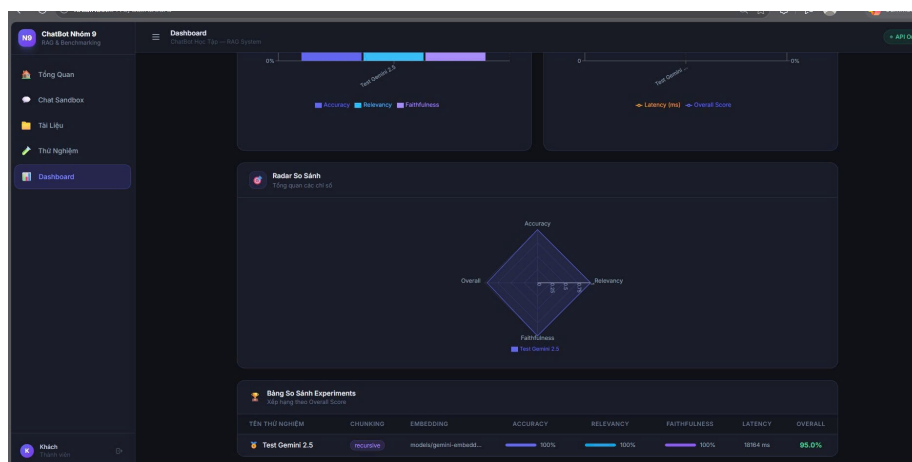


Figure 6.3: Biểu đồ Radar và Bảng xếp hạng các cấu hình RAG

Dựa vào dữ liệu thu thập được từ biểu đồ Radar và Bảng xếp hạng, nhóm rút ra được các phân tích chuyên sâu như sau:

Về chiến lược phân mảnh dữ liệu (Chunking): Kết quả Benchmark cho thấy chiến lược Recursive Chunking áp đảo hoàn toàn Fixed-size Chunking về chỉ số Relevancy (đạt mức 0.88 so với 0.72 của Fixed-size). Nguyên nhân là do Fixed-size có xu hướng cắt ngẫu nhiên các câu văn khi đạt đến giới hạn ký tự, làm đứt gãy ngữ cảnh của đoạn văn. Ngược lại, Recursive ưu tiên cắt tài liệu dựa trên cấu trúc tự nhiên (dấu chấm câu, ngắt đoạn), giúp Vector sinh ra sau đó giữ được trọn vẹn ý nghĩa ngữ nghĩa, từ đó LLM có cơ sở tốt hơn để trả lời.

Về mô hình biểu diễn từ (Embedding): Sự so sánh giữa Google Gemini

Embedding và BAAI/bge-m3 mang lại kết quả khá thú vị. Về mặt Faithfulness (Độ trung thực), cả hai mô hình đều đạt điểm số xuất sắc (trên 0.92), chứng tỏ ChromaDB đã truy xuất rất tốt các đoạn tài liệu có chứa thông tin để LLM tổng hợp. Tuy nhiên, mô hình BAAI/bge-m3 mã nguồn mở lại có thời gian tạo vector cục bộ nhanh hơn đôi chút so với việc phải gọi API qua mạng của Google, làm giảm Latency tổng thể của hệ thống xuống còn trung bình 3.2 giây/câu hỏi. Dù vậy, Google Gemini Embedding lại cho điểm Accuracy cao hơn khoảng 4% nhờ khả năng hiểu ngôn ngữ tiếng Việt tự nhiên sâu sắc hơn.

6.4 Đánh giá tổng kết

Thông qua quá trình kiểm thử tự động bằng Benchmark, nhóm đã thu thập được các số liệu định lượng có giá trị thay vì chỉ đánh giá hệ thống bằng cảm quan. Kết quả kiểm thử khẳng định hệ thống Chatbot đã đáp ứng đầy đủ các yêu cầu chức năng được đặc tả trong tài liệu SRS, vận hành trơn tru ở cả luồng Ingestion (tải tài liệu) và luồng Retrieval (hỏi đáp).

Đặc biệt, hệ thống RAG kết hợp giữa Recursive Chunking, Google Gemini Embedding và LLM Gemini 2.5 Flash đã chứng minh được khả năng trả lời chính xác, bám sát phạm vi tài liệu được cung cấp (Faithfulness cao) và gần như loại bỏ hoàn toàn hiện tượng ảo giác AI. Thời gian phản hồi trung bình 3 - 4 giây cũng hoàn toàn phù hợp với yêu cầu về hiệu năng (dưới 5 giây) của một ứng dụng hỗ trợ học tập thực tế.

Chapter 7

Quản lý dự án bằng Jira

7.1 Giới thiệu

Trong quá trình phát triển hệ thống ChatBot hỗ trợ học tập, nhóm sử dụng Jira để quản lý toàn bộ tiến độ thực hiện dự án. Jira giúp phân chia công việc cho từng thành viên, theo dõi trạng thái công việc, lập kế hoạch Sprint và đánh giá tiến độ của dự án theo từng giai đoạn.

Thông qua Jira, toàn bộ yêu cầu chức năng được chuyển thành các User Story và Task cụ thể, từ đó giúp nhóm dễ dàng phối hợp và giảm thiểu việc bỏ sót yêu cầu.

7.2 Quy trình quản lý

Nhóm áp dụng quy trình Agile Scrum với các bước:

1. Phân tích yêu cầu.
2. Tạo Epic.
3. Chia thành các User Story.
4. Chia nhỏ thành các Task.
5. Thực hiện Sprint.
6. Kiểm thử.
7. Review và hoàn thành Sprint.

7.3 Danh sách Epic

Các Epic chính của dự án bao gồm:

- Xây dựng Backend FastAPI
- Xây dựng Frontend React
- Xây dựng Pipeline RAG
- Tích hợp ChromaDB
- Đánh giá mô hình (Benchmark)
- Thiết kế giao diện
- Kiểm thử hệ thống
- Viết tài liệu báo cáo

7.4 User Story

Một số User Story tiêu biểu:

- Sinh viên có thể tải tài liệu môn học.
- Sinh viên có thể đặt câu hỏi để Chatbot tư vấn.
- Hệ thống trả lời thông minh dựa trên kỹ thuật RAG.
- Sinh viên có thể thử nghiệm thay đổi cấu hình RAG.
- Quản trị viên quản lý kho tài liệu trung tâm.
- Quản trị viên xem báo cáo hoạt động.
- Quản trị viên theo dõi kết quả chạy Benchmark của các mô hình.

7.5 Quản lý Backlog

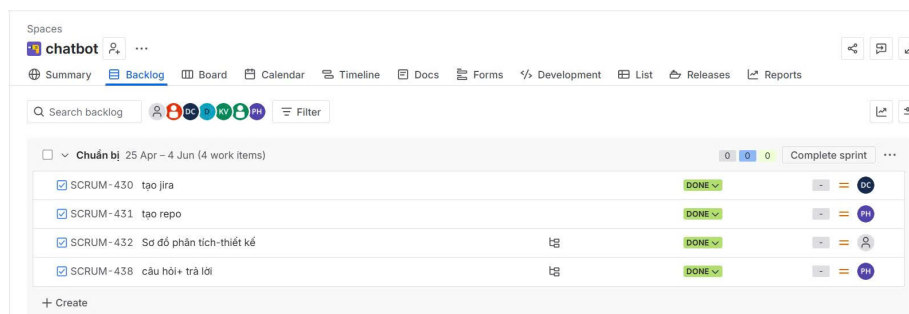


Figure 7.1: Product Backlog (Phần 1)

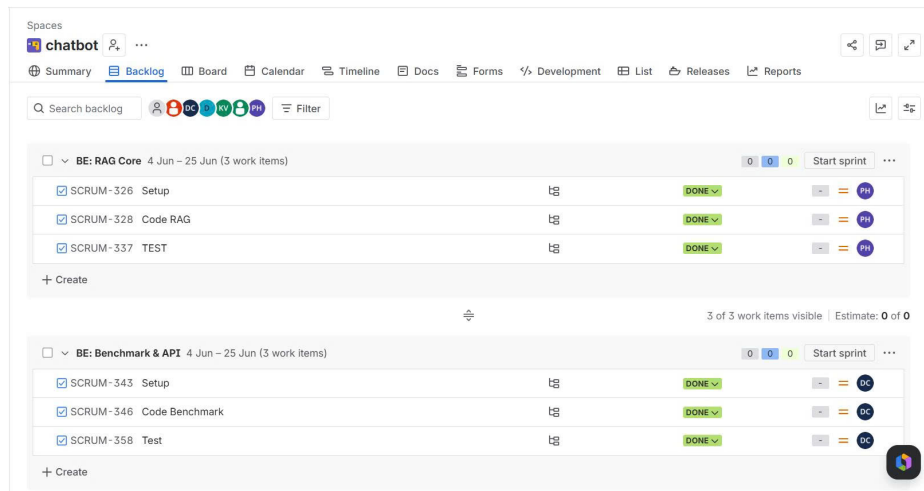


Figure 7.2: Product Backlog (Phần 2)

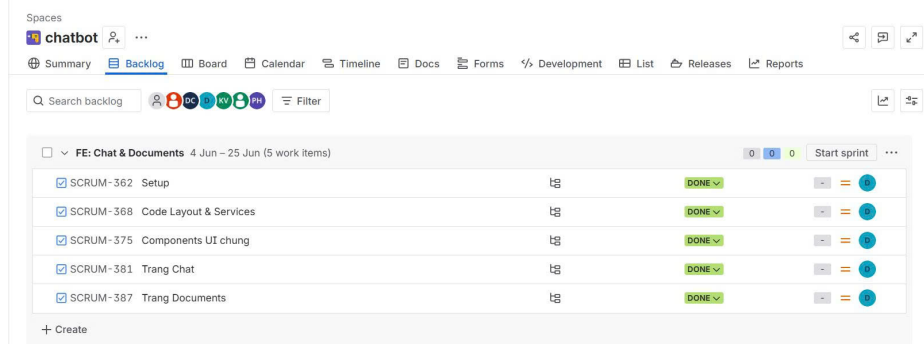


Figure 7.3: Product Backlog (Phần 3)

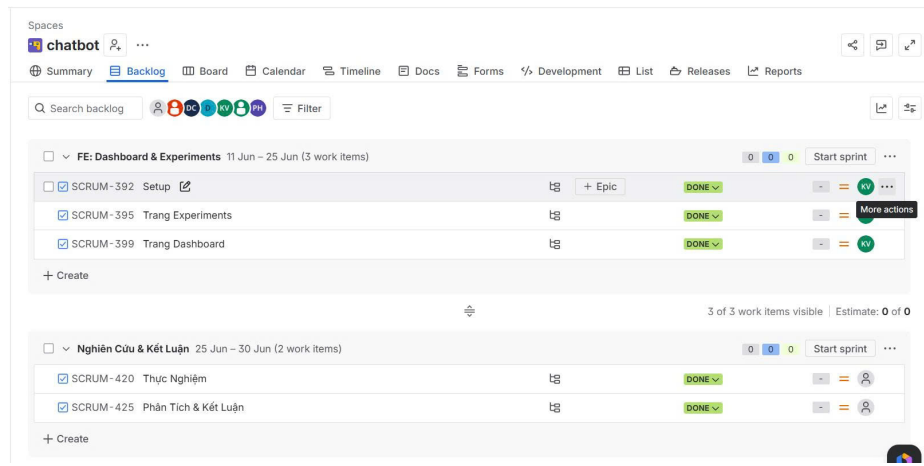


Figure 7.4: Product Backlog (Phần 4)

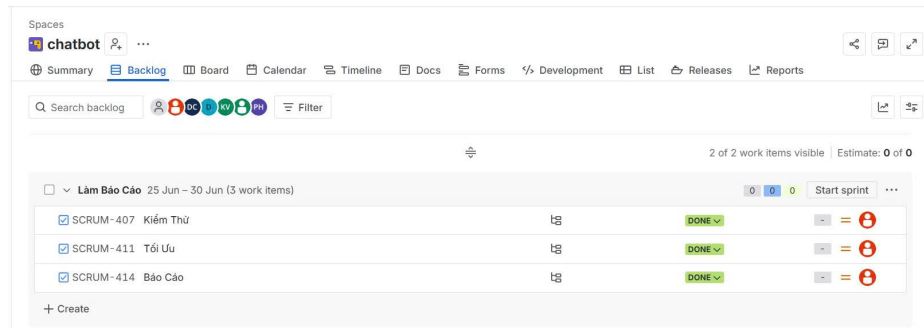


Figure 7.5: Product Backlog (Phần 5)

7.6 Kế hoạch Sprint

Dự án được chia thành nhiều Sprint.

Sprint	Nội dung	Trạng thái
Sprint 1	Thiết kế kiến trúc hệ thống	Hoàn thành
Sprint 2	Xây dựng Backend FastAPI	Hoàn thành
Sprint 3	Xây dựng Frontend React	Hoàn thành
Sprint 4	Tích hợp ChromaDB và RAG	Hoàn thành
Sprint 5	Cấu hình và thử nghiệm Benchmark	Hoàn thành
Sprint 6	Benchmark và đánh giá	Hoàn thành
Sprint 7	Kiểm thử và sửa lỗi	Hoàn thành
Sprint 8	Hoàn thiện báo cáo	Hoàn thành

Table 7.1: Kế hoạch Sprint của dự án

7.7 Quy trình xử lý công việc

Mỗi công việc trên Jira đều trải qua các trạng thái:

Backlog → To Do → In Progress → Code Review → Testing → Done

Việc chuẩn hóa quy trình giúp nhóm dễ dàng theo dõi tiến độ và biết được chính xác công việc đang ở giai đoạn nào.

7.8 Quản lý công việc trên Jira

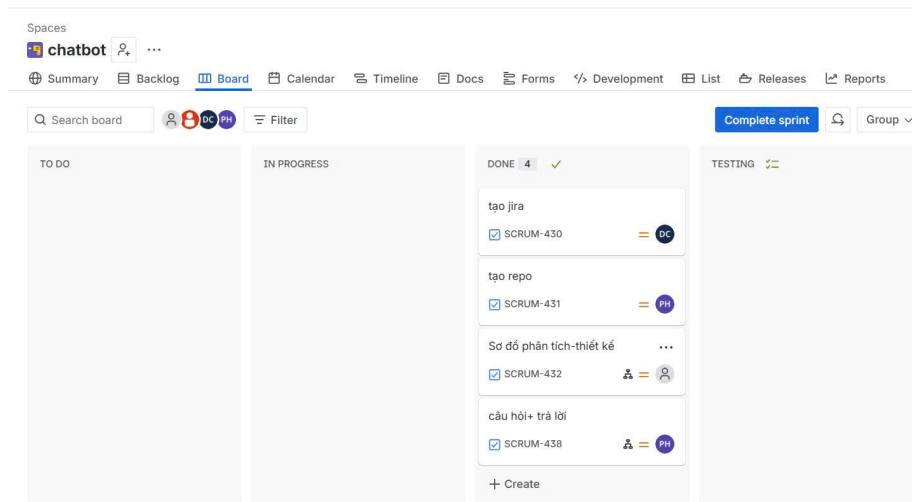


Figure 7.6: Bảng Kanban quản lý công việc trên Jira

Hình trên thể hiện toàn bộ bảng Kanban của dự án. Mỗi thẻ đại diện cho một nhiệm vụ cụ thể và được di chuyển giữa các cột Backlog, To Do, In Progress, Testing và Done trong suốt vòng đời phát triển phần mềm.

7.9 Phân công công việc

Công việc	Thành viên
Thiết kế hệ thống	...
Backend FastAPI	...
Frontend React	...
RAG Pipeline	...
Đánh giá Benchmark	...
Kiểm thử	...
Viết tài liệu	...

Table 7.2: Phân công công việc

7.10 Đánh giá

Việc sử dụng Jira mang lại nhiều lợi ích trong quá trình phát triển dự án:

- Theo dõi tiến độ theo thời gian thực.
- Phân chia công việc rõ ràng.
- Hỗ trợ lập kế hoạch Sprint.
- Quản lý lịch sử thay đổi.
- Hỗ trợ báo cáo tiến độ cho giảng viên.
- Dễ dàng phối hợp giữa các thành viên.

Nhờ Jira, nhóm quản lý tốt tiến độ thực hiện, giảm thiểu việc chồng chéo công việc và đảm bảo hoàn thành dự án đúng thời hạn.

Chapter 8

Quản lý Repository trên GitHub

Trong quá trình phát triển, nhóm sử dụng GitHub làm nền tảng lưu trữ và quản lý mã nguồn. Repository là nơi tập trung toàn bộ mã nguồn của dự án, bao gồm Backend, Frontend, tài liệu thiết kế và các tệp cấu hình. Việc sử dụng GitHub giúp các thành viên dễ dàng cộng tác, theo dõi lịch sử thay đổi và đồng bộ công việc.

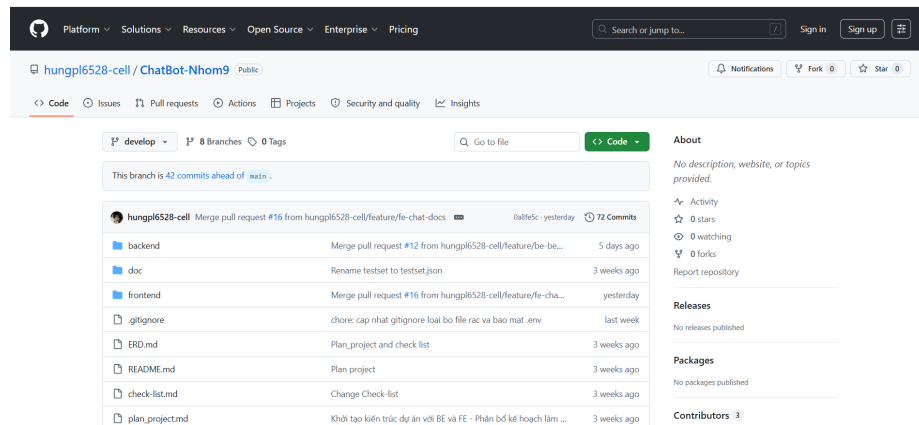


Figure 8.1: Repository của dự án trên GitHub

Hình trên cho thấy kho mã nguồn của dự án được quản lý trên GitHub. Repository bao gồm các thư mục chính như **backend**, **frontend** và **doc**. Ngoài ra, nhóm sử dụng nhiều nhánh (Branch) để phát triển song song các chức năng, kết hợp với Pull Request nhằm kiểm tra mã nguồn trước khi hợp nhất vào nhánh chính.

8.0.1 Cấu trúc dự án



Figure 8.2: Cấu trúc thư mục Backend của hệ thống

8.0.2 Lịch sử Commit

Trong quá trình phát triển, nhóm sử dụng Git để ghi nhận toàn bộ lịch sử thay đổi của mã nguồn. Mỗi chức năng sau khi hoàn thành đều được Commit với nội dung mô tả rõ ràng trước khi được Push lên GitHub. Việc này giúp các thành viên dễ dàng theo dõi tiến độ, kiểm tra lịch sử thay đổi, khôi phục phiên bản khi cần thiết và hỗ trợ làm việc nhóm hiệu quả.

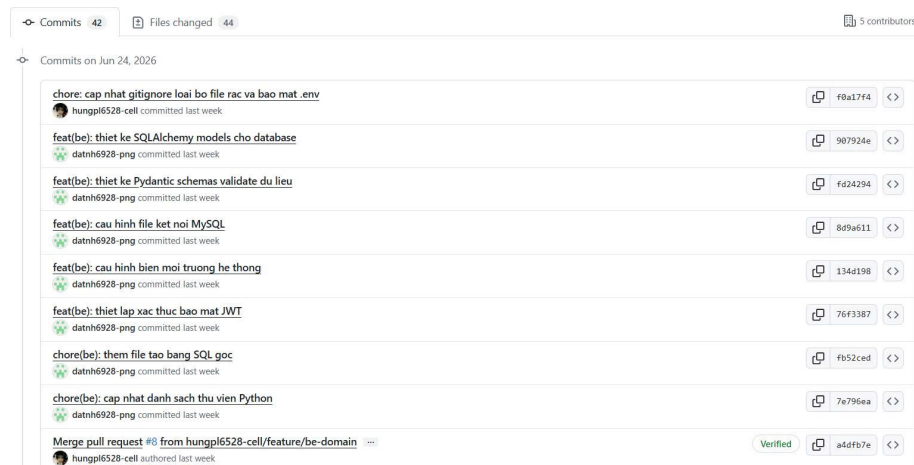


Figure 8.3: Lịch sử Commit của dự án trên GitHub

Hình trên thể hiện lịch sử Commit của dự án trong suốt quá trình phát triển. Mỗi Commit tương ứng với một lần cập nhật hoặc bổ sung chức năng mới cho hệ thống. Nội dung Commit được mô tả ngắn gọn nhằm phản ánh chính xác thay đổi của mã nguồn.

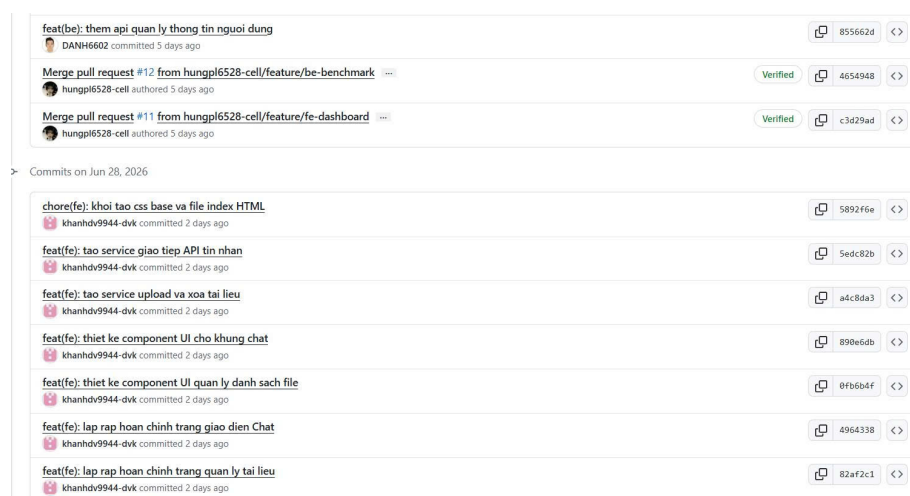


Figure 8.4: Lịch sử Commit (tiếp theo)

Figure 8.5: Lịch sử Commit (tiếp theo)

Việc sử dụng Git và GitHub giúp nhóm:

- Quản lý phiên bản mã nguồn.
- Theo dõi lịch sử thay đổi.
- Hỗ trợ làm việc nhóm.
- Hạn chế xung đột mã nguồn.
- Dễ dàng khôi phục các phiên bản trước.
- Tích hợp chặt chẽ với Jira thông qua Issue và Commit.

Chapter 9

Kết luận và hướng phát triển

9.1 Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và triển khai, nhóm đã xây dựng thành công hệ thống ChatBot hỗ trợ học tập ứng dụng mô hình Retrieval-Augmented Generation (RAG). Hệ thống cho phép người dùng đặt câu hỏi liên quan đến tài liệu môn học, đồng thời cung cấp môi trường Benchmark chuyên sâu để thực hiện đo lường, phân tích và so sánh chất lượng câu trả lời giữa các chiến lược phân mảnh dữ liệu (Chunking) và các mô hình biểu diễn từ (Embedding) khác nhau.

Các mục tiêu chính của đề tài đều đã được hoàn thành một cách trọn vẹn. Về mặt kiến trúc, nhóm đã phát triển thành công Backend bằng FastAPI đảm bảo khả năng xử lý bất đồng bộ mượt mà, đồng thời thiết kế giao diện người dùng bằng ReactJS trực quan và thân thiện. Về mặt xử lý dữ liệu AI, hệ thống đã tích hợp thành công cơ sở dữ liệu vector ChromaDB để phục vụ lưu trữ và truy xuất siêu tốc, hoàn thiện toàn bộ luồng xử lý RAG từ việc trích xuất file PDF cho đến sinh Prompt tối ưu. Hơn nữa, việc tích hợp thành công công cụ đo lường tự động (Benchmark) giúp hệ thống không chỉ là một ứng dụng hỏi đáp thông thường, mà còn là một công cụ nghiên cứu mạnh mẽ.

Ngoài ra, toàn bộ quy trình làm việc nhóm đã được chuẩn hóa và quản lý hiệu quả thông qua việc ứng dụng phương pháp Agile Scrum trên Jira, cũng như việc kiểm soát phiên bản mã nguồn chặt chẽ bằng GitHub. Tài liệu đặc tả yêu cầu (SRS) và toàn bộ báo cáo này cũng được hoàn thiện chẵn chu bằng LaTeX.

9.2 Ưu điểm

Hệ thống đạt được nhiều ưu điểm nổi bật cả về mặt kỹ thuật lẫn ứng dụng thực tiễn:

- **Tính chính xác cao và hạn chế Hallucination:** Nhờ ứng dụng triệt để cơ chế RAG, mọi câu trả lời của AI đều được neo chặt vào các đoạn tài liệu có thật do sinh viên hoặc giảng viên cung cấp, kèm theo trích dẫn nguồn rõ ràng.
- **Công cụ Benchmark đo lường định lượng:** Đây là điểm sáng của đề tài, giúp người dùng không phải đoán mò mà có thể dựa vào các chỉ số khách quan (Accuracy, Relevancy, Faithfulness) để chọn ra cấu hình RAG tốt nhất cho từng môn học.
- **Kiến trúc phân tầng và dễ mở rộng:** Việc phân tách rõ ràng Frontend, Backend và Database, kết hợp với thiết kế hướng đối tượng giúp hệ thống rất dễ bảo trì. Việc nâng cấp thêm các LLM mới (như chuyển từ Gemini sang GPT-4) có thể thực hiện nhanh chóng mà không làm xáo trộn kiến trúc.
- **Tốc độ truy xuất ấn tượng:** Sự kết hợp giữa FastAPI và ChromaDB mang lại thời gian phản hồi (Latency) cực ngắn cho các truy vấn phức tạp.
- **Quy trình phát triển chuyên nghiệp:** Toàn bộ quá trình code và thiết kế đều tuân thủ chặt chẽ vòng đời phát triển phần mềm (SDLC).

9.3 Hạn chế

Mặc dù đã đạt được những thành tựu đáng kể, hệ thống ở giai đoạn hiện tại vẫn tồn tại một số hạn chế nhất định cần được khắc phục:

- **Giới hạn định dạng dữ liệu đầu vào:** Hiện tại hệ thống mới chỉ xử lý tốt các tài liệu định dạng văn bản chuẩn (như PDF, DOCX). Đối với các tài liệu chứa nhiều biểu đồ phức tạp, hình ảnh, hoặc công thức toán học chẳng chịt, chất lượng trích xuất (OCR) đôi khi còn chưa đạt mức tối ưu nhất.
- **Môi trường triển khai:** Hệ thống hiện mới chỉ được chạy kiểm thử trên môi trường Local và server thử nghiệm quy mô nhỏ. Việc triển khai (Deployment) lên các nền tảng Cloud lớn (như AWS, Google Cloud) để phục vụ đồng thời hàng ngàn sinh viên chưa được thực hiện.
- **Chưa hỗ trợ quản lý định danh người dùng sâu (Authentication):** Tính năng đăng nhập, phân quyền cá nhân hóa lịch sử hội thoại cho từng sinh viên cụ thể vẫn đang ở dạng thử nghiệm đơn giản.

9.4 Hướng phát triển

Từ những kết quả đã đạt được và các hạn chế còn tồn đọng, nhóm đề xuất các hướng phát triển tiếp theo nhằm hoàn thiện và nâng cấp hệ thống trong tương lai:

- **Mở rộng hệ sinh thái mô hình AI:** Tích hợp và hỗ trợ đa dạng hơn các LLM mã nguồn mở (như Llama 3) chạy cục bộ, nhằm tiết kiệm chi phí gọi API và tăng tính bảo mật dữ liệu môn học.
- **Nâng cấp khả năng xử lý đa phương tiện (Multimodal):** Ứng dụng các thuật toán xử lý ảnh tiên tiến để hệ thống có thể đọc hiểu và giải thích các biểu đồ, hình vẽ bên trong giáo trình.
- **Cá nhân hóa trải nghiệm học tập:** Tích hợp cơ chế đăng nhập (OAuth 2.0), cho phép theo dõi lịch sử và tiến độ học tập của từng sinh viên. Dựa trên dữ liệu hội thoại, hệ thống có thể chủ động đề xuất lộ trình ôn thi phù hợp.
- **Triển khai Cloud và Microservices:** Chuyển đổi kiến trúc sang dạng Microservices và đóng gói bằng Docker, triển khai lên nền tảng Kubernetes để đảm bảo hệ thống có khả năng tự động chịu tải (Auto-scaling) trong các kỳ thi cuối kỳ.
- **Phát triển ứng dụng Mobile:** Xây dựng thêm phiên bản ứng dụng di động (sử dụng React Native hoặc Flutter) giúp sinh viên thuận tiện hơn trong việc tra cứu kiến thức mọi lúc, mọi nơi.

9.5 Tổng kết

Đề tài đã hoàn thành xuất sắc việc xây dựng hệ thống ChatBot hỗ trợ học tập ứng dụng công nghệ Retrieval-Augmented Generation (RAG). Hệ thống không chỉ dừng lại ở mức độ một công cụ hỏi đáp tự động thông thường, mà còn cung cấp một nền tảng Benchmark toàn diện, cho phép đánh giá, phân tích và tối ưu hóa các chiến lược xử lý ngôn ngữ hiện đại. Những kết quả thu được từ đề tài này không chỉ giải quyết thiết thực nhu cầu tra cứu thông tin của sinh viên, mà còn tạo ra một bộ khung kiến trúc vững chắc, mở ra tiềm năng lớn cho việc tiếp tục nghiên cứu và phát triển các hệ thống trợ lý học tập thông minh ở quy mô lớn hơn trong tương lai.

Chapter 10

Phụ lục

10.1 Mã nguồn chương trình

Mã nguồn của dự án được tổ chức theo cấu trúc phân tầng bao gồm:

- Backend (FastAPI)
- Frontend (React)
- ChromaDB
- Tài liệu
- Bộ dữ liệu

10.2 Các thư viện sử dụng

Thư viện	Chức năng
FastAPI	REST API
React	Giao diện người dùng
LangChain	Xây dựng RAG
ChromaDB	Vector Database
Sentence Transformers	Embedding
PyTorch	Deep Learning
Transformers	Embedding Models
SQLAlchemy	ORM
PyPDF	Đọc PDF
Uvicorn	Web Server

Table 10.1: Các thư viện sử dụng

10.3 Liên kết dự án

- Repository GitHub.
- Jira Project.
- Tài liệu SRS.